

YAZILIM TEST MÜHENDİSLİĞİ

Modern Test Mimarisi

UMUT BELER

22360859015

YAZILIM TEST MÜHENDİSLİĞİ KAVRAMI

Yazılım Test Mühendisliği, geliştirilen bir uygulamanın beklentilere tam uygun, hatasız ve güvenilir çalıştığını garanti altına alan teknik bir disiplindir. Temel amacı yalnızca var olan hataları (bug) aramak değil; sistemin kendi kendini test edebileceği otomatik altyapılar ve kalıcı çözümler inşa etmektir.

SDET TANIMI

Software Development Engineer in Test: Kod yazma yetkinliğine sahip, test framework'lerini bir yazılım geliştiricisi gibi inşa eden mühendislik rolüdür.

TEKNİK TEST SEVİYELERİ

BİRİM (UNIT) TESTLERİ

Kodun en küçük parçalarının (fonksiyon, metod) mantıksal doğruluğunun izole edilerek test edilmesi.

ENTEGRASYON TESTLERİ

Farklı modüllerin ve servislerin bir araya geldiğinde birbirleriyle olan veri alışverişinin doğrulanması.

SİSTEM (END-TO-END)

Uygulamanın tüm bağımlılıkları ile birlikte, kullanıcı senaryolarına dayalı uçtan uca test edilmesi.

TEST AŞAMALARI

PLANLAMA

Test stratejisi ve araç seçimi.

KOŞUM

Otomasyon ve manuel
yürütme.

ANALİZ

Gereksinimlerin
incelemesi.

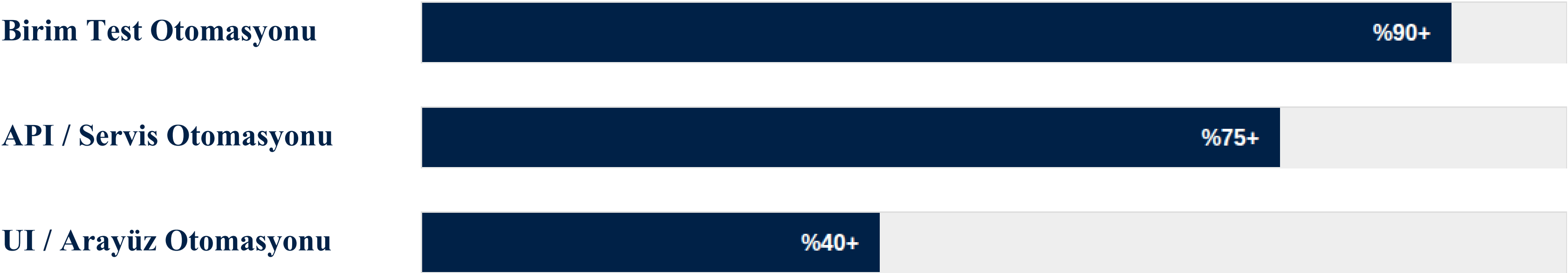
TASARIM

Test senaryoları ve veri
hazırlığı.

DEĞERLENDİRME

Raporlama ve kriter kontrolü.

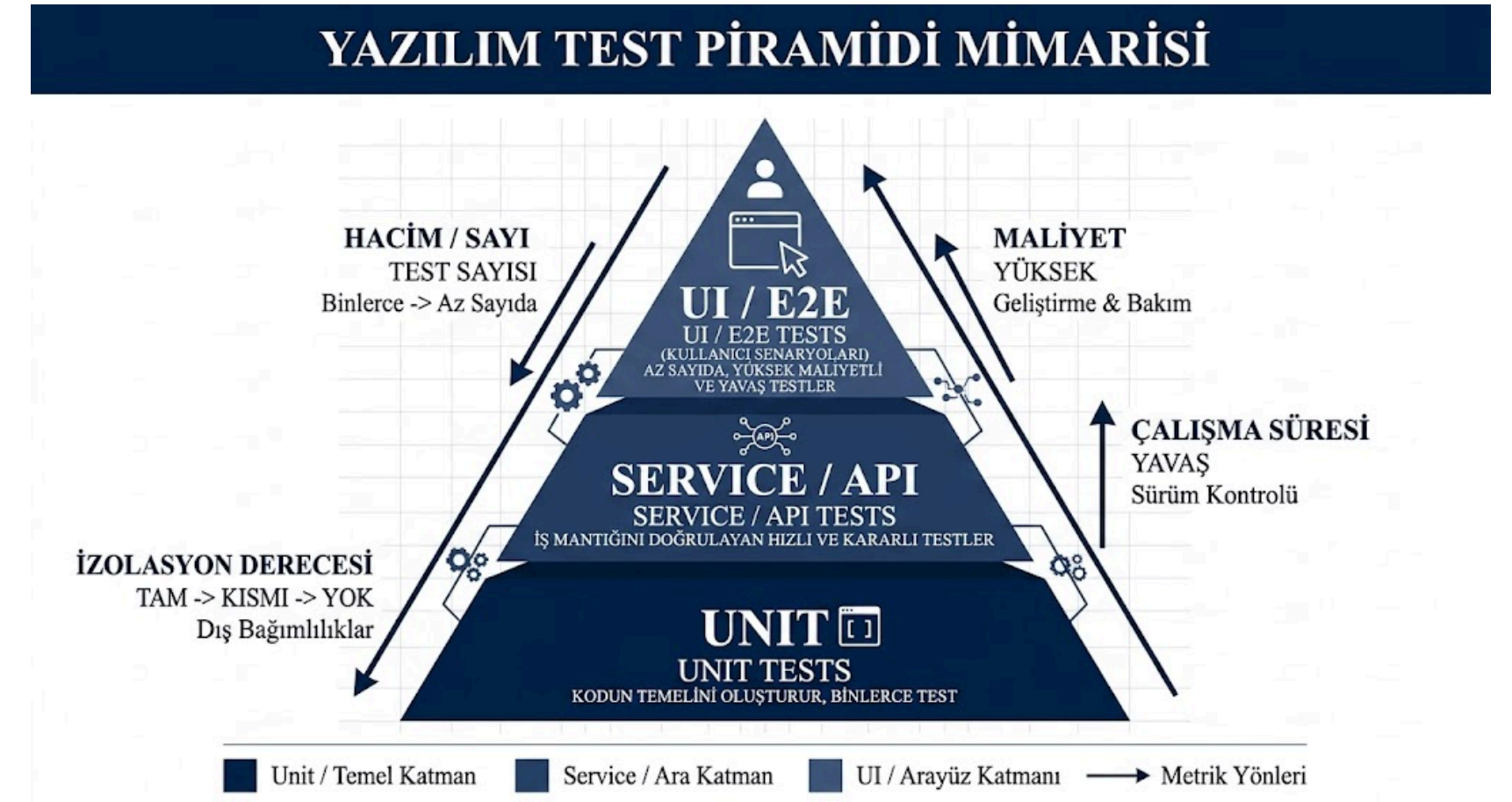
OTOMASYON STRATEJİSİ VE ROI



Teknik Borç oluşumunu engellemek için regresyon testlerinin otomasyon oranı kritik önem taşır.

TEST PİRAMİDİ MİMARİSİ

- **UI / E2E:** Az sayıda, yüksek maliyetli ve yavaş testler.
- **Service / API:** İş mantığını doğrulayan hızlı ve kararlı testler.
- **Unit:** Kodun temelini oluşturan, en hızlı ve binlerce sayıdaki testler.



API VE VERİ TABANI TEST MÜHENDİSLİĞİ

API SÖZLEŞME TESTLERİ

Endpoint'lerin şema doğruluğu, HTTP durum kodları ve JSON/XML veri yapılarının teknik doğrulanması.

VERİ BÜTÜNLÜĞÜ

Veri tabanı (CRUD) işlemlerinin, tetikleyicilerin ve saklı yordamların (Stored Procedures) teknik doğrulanması.

SÜREKLİ TEST

Sürekli test; yazılım testinin "sondaki bir kontrol aşaması" olmaktan çıkıp, kodlama anından canlıya (production) alınana kadar kesintisiz devam eden otonom bir sürece dönüşmesidir.

Pipeline Entegrasyonu Adımları:

- **Tetiklenme (Push):** Geliştiricinin yeni kodu merkezi depoya(Git) göndermesiyle otomasyon sürecinin başlaması.
- **Statik Kod Analizi:** Kodun çalıştırılmadan önce okunarak; güvenlik açıklarının, kodlama hatalarının ve standart ihlallerinin tespit edilmesi.
- **İzole Ortamda Koşum:** Uygulamanın sanal bir ortamda (Örn: Docker) ayağa kaldırılarak birim (Unit) ve entegrasyon testlerinin otomatik çalıştırılması.
- **Kalite Kapıları (Quality Gates):** "Test kapsamı (coverage) %80'in altındaysa" veya "Kritik bir hata bulunduysa" kodun canlıya çıkışını engelleyen otomatik bariyer mekanizması.

TEKNİK ARAÇ EKOSİSTEMİ

UI OTOMASYON

Selenium, Playwright,
Cypress, Appium.

API & PERFORMANS

Postman, RestAssured,
JMeter, k6.

ALTYAPI & CI

Docker, Jenkins, GitLab CI,
SonarQube.

TEST MÜHENDİSLİĞİNDE YAPAY ZEKA ETKİSİ

Yapay zeka, geleneksel kural tabanlı (rule-based) test otomasyonundan; büyük dil modelleri ve makine öğrenmesi yöntemleriyle desteklenen, adaptif ve otonom test sistemlerine geçişi hızlandırmaktadır . Bu yeni nesil mimari, statik kod yazımını en aza indirerek sistemin kendi kendini test etme kapasitesini maksimize etmeyi amaçlar.

Yapay zekanın test mühendisliğinde kullanım senaryoları örnekleri:

- Kendi kendini onaran otomasyon
- Otonom test senaryosu ve kod üretimi
- Gözlemlenebilirlik ve telemetri
- Tahminleyici analitik ile optimizasyon

GELECEĞİN TEST ÇÖZÜMLERİ

- **Kendi Kendini Onaran Otomasyon:** Geliştirici arayüzde bir butonun ID'sini, CSS sınıfını veya XPATH konumunu değiştirdiğinde testin patlamasını önleyen sistemlerdir. Yapay zeka, sayfanın DOM ağacını ve görsel yapısını analiz ederek değişen bileşeni dinamik olarak bulur ve test kodunu çalışma anında otonom olarak günceller.
- **Otonom Test Senaryosu ve Kod Üretimi:** Yapay Zeka modelleri kullanılarak; kullanıcı hikayelerinden, API dokümantasyonlarından veya doğrudan arayüz analizinden otomatik olarak test senaryoları çıkarılmasını sağlayan sistemlerdir. Sistem, belirlenen uç durumları hesaplar ve Cypress, Playwright gibi araçlar için çalıştırılabilir test kodlarını kendi yazar.

- **Gözlemlenebilirlik ve Telemetry:** Sistemin sadece "Geçti/Kaldı" mantığıyla değil, canlı ortamdaki log, metrik ve iz verileriyle test edilmesidir. Makine öğrenmesi algoritmaları, canlı sistem ortamındaki veri akışını izleyerek anormallikleri tespit eder ve hataları son kullanıcı şikayet etmeden önce proaktif olarak raporlar.
- **Tahminleyici Analitik ile Optimizasyon:** Geçmiş Git commit geçmişini, kod değişikliklerini ve hata raporlarını analiz eden yapay zeka modelleri, bir sonraki sürümde uygulamanın hangi modüllerinin patlama riskinin yüksek olduğunu hesaplar. Böylece binlerce test koşturmak yerine sadece en riskli alanların test edilmesi sağlanarak süreç hızlandırılır.

TEŞEKKÜRLER